

Memora

A Project report submitted in partial fulfillment of the requirements for the award of degree in

BACHELOR OF TECHNOLOGY

(COMPUTER SCIENCE AND ENGINEERING)

SUBMITTED BY

Registration number	Name of the Student
A22126510134	Bheesetti Harsith Veera Charan
A22126510144	D. Chaitanya
A22126510163	M. Sai Teja
A22126510193	Venkata Vishaal Tirupalli
A22126510194	Wuna Akhilesh

UNDER THE GUIDANCE OF

Dr. D. Naga Teja

Associate professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES (A+)

SANGIVALASA, VISAKHAPATNAM – 531162

July - 2025

Memora

For Spaced Repetition Learning based on Ebbinghaus Forgetting Curve

Technical Documentation and Report

Version 1.0 • 2026-03-28

Learning Smarter • Retaining Longer • Performing Better

Powered by Smart Repetition

Contents

1	Introduction	5
1.1	Motivation and Context	5
1.2	Problem Statement	5
1.3	Objectives	6
2	Literature Review	8
2.1	Spaced Repetition Research	8
2.2	Cognitive Assessment Frameworks	9
2.3	User Interface and Learning UX	10
2.4	Technical Implementation Considerations	10
3	System Design	12
3.1	System Architecture	12
3.2	Database Schema Design	13
3.3	API Endpoint Specifications	14
3.4	Entity-Relationship Diagrams	16
4	Implementation Details	18
4.1	SM-2 Spaced Repetition Algorithm	18
4.2	Crowding Prevention Algorithm	19
4.3	MemScore Cognitive Assessment	20
4.4	Authentication System	21
4.5	Frontend Component Architecture	21
5	Testing Considerations	23
5.1	Unit Testing Areas	23
5.2	Integration Testing	23
5.3	Performance Metrics	24
6	Deployment Considerations	25
6.1	Environment Configuration	25
6.2	Security Hardening	25
6.3	Scalability Considerations	25
7	Future Enhancements	26
7.1	Planned Features	26
7.2	Technical Improvements	26
8	Conclusion	27
9	Gallery	28
10	References	31
11	Appendix A: Technology Stack Summary	32
11.1	Frontend Technologies	32
11.2	Backend Technologies	32
11.3	Development Tools	32
12	Appendix B: Database Schema Reference	33
12.1	User Collection Schema	33
12.2	Topic Collection Schema	33
12.3	Database Indexes	34
13	Appendix C: Project Statistics	35
13.1	Codebase Metrics	35

13.2 Feature Coverage	35
-----------------------------	----

1 Introduction

Memora is an advanced spaced repetition learning platform designed to optimize memory retention through scientifically-backed algorithms and personalized cognitive assessment. The system combines modern web technologies with proven learning methodologies to create an intelligent, adaptive learning environment that helps users retain information more effectively and study smarter.

The name “Memora” derives from the concept of “memory in motion,” emphasizing the platform’s dynamic approach to knowledge retention. Rather than static flashcards or passive review systems, Memora actively adapts to each user’s cognitive profile and learning patterns, creating a personalized learning journey that maximizes retention while minimizing study time waste.

1.1 Motivation and Context

Traditional learning methods often suffer from inefficient review schedules and lack of personalization. Research in cognitive science has consistently shown that spaced repetition—the practice of reviewing information at increasing intervals—is one of the most effective techniques for long-term memory retention. However, implementing this technique manually is cumbersome and error-prone.

The forgetting curve, first described by Hermann Ebbinghaus in 1885, demonstrates that memory retention decreases exponentially over time. Without reinforcement, newly learned information can be forgotten within days or even hours. Ebbinghaus’s research showed that the rate of forgetting follows a predictable pattern, but strategic review at specific intervals can dramatically improve long-term retention.

Despite this well-established scientific foundation, most students and professionals still rely on inefficient study methods such as cramming, re-reading notes without strategic timing, or using generic study apps that don’t account for individual differences. This disconnect between what cognitive science tells us works and what people actually do in practice represents a significant opportunity for technological intervention.

Memora addresses this challenge by automating the spaced repetition process while adding sophisticated cognitive assessment capabilities. The platform evaluates each user’s learning profile through comprehensive testing, then adapts the review schedule based on individual performance patterns and cognitive strengths. By combining the SuperMemo SM-2 algorithm with modern web technologies and user-centered design, Memora makes evidence-based learning accessible and practical for everyday use.

1.2 Problem Statement

Students and lifelong learners face several critical challenges in effective knowledge retention:

1. **Inefficient Review Scheduling:** Manual tracking of when to review topics leads to forgotten material or premature reviews. Users struggle to determine optimal review timing, often reviewing too early (wasting time) or too late (requiring complete relearning). Without

algorithmic support, maintaining an effective spaced repetition schedule for multiple topics becomes overwhelming.

2. **One-Size-Fits-All Approaches:** Generic learning apps don't account for individual cognitive differences and learning patterns. A student with strong visual memory may have different optimal review intervals than one with stronger verbal processing. Current platforms apply the same scheduling algorithm to all users regardless of their cognitive profile, leading to suboptimal outcomes.
3. **Lack of Performance Insights:** Users cannot easily identify their cognitive strengths and areas needing improvement. Without baseline cognitive assessment, learners don't know whether poor performance stems from inadequate study time, ineffective study methods, or cognitive factors. This lack of insight prevents targeted improvement strategies.
4. **Poor Study Organization:** Difficulty managing multiple subjects, topics, and review schedules simultaneously. As the number of topics grows, manual scheduling becomes impractical. Users need intelligent load balancing to prevent overwhelming study days while ensuring consistent progress across all learning materials.
5. **Motivation Issues:** Without proper progress tracking and personalized feedback, learners often abandon study routines. The abstract nature of memory improvement makes it difficult to perceive progress, leading to decreased motivation over time. Gamification elements and visual progress indicators are needed to maintain long-term engagement.
6. **Information Overload:** In an era where information is abundant but attention is scarce, learners struggle to focus on what truly needs review. Smart filtering and prioritization based on actual forgetting patterns rather than arbitrary schedules is essential for efficient learning.
7. **Lack of Integration:** Existing solutions often separate cognitive assessment from learning content management, forcing users to maintain multiple tools and manually correlate their performance data with study materials.

1.3 Objectives

The main objectives of this project are:

1. **Implement Intelligent Spaced Repetition:** Develop an enhanced SM-2 algorithm that adapts to individual learning patterns and performance. The system should automatically calculate optimal review intervals based on difficulty level, user performance, and cognitive profile. Implementation includes ease factor adjustments, interval calculations, and intelligent rescheduling to prevent crowded study days.
2. **Create Comprehensive Cognitive Assessment:** Design and implement a multi-faceted evaluation system (MemScore) that measures memory capacity, processing speed, and spatial reasoning. The assessment should be completable in 10-15 minutes while providing reliable indicators of cognitive strengths and learning capacity. Results should be quantified on a 0-100 scale and used to personalize the learning algorithm.
3. **Build Modern Web Interface:** Develop a responsive, cyber-themed user interface that provides an engaging and distraction-free learning experience. The interface should feature a dark-theme cyber-grid aesthetic with neon accents, smooth animations powered by Framer Motion, and responsive design optimized for both desktop and mobile devices. The design should minimize cognitive load while maximizing focus on learning content.

4. **Ensure Robust Data Management:** Implement secure user authentication with JWT tokens and refresh token rotation, comprehensive data persistence in MongoDB, and reliable performance tracking across all user interactions. The system should maintain data integrity while providing fast access to user profiles, topics, and performance metrics.
5. **Provide Actionable Insights:** Offer detailed analytics and progress tracking to help users understand their learning patterns and optimize study strategies. This includes visualization of retention rates, identification of difficult topics, tracking of study streaks, and personalized recommendations for improving learning efficiency.
6. **Enable Scalable Architecture:** Design the system with scalability in mind, using a modern MERN stack (MongoDB, Express.js, React, Node.js) that can handle growing user bases and expanding feature sets. The architecture should support future enhancements such as collaborative learning, mobile applications, and advanced analytics.
7. **Maintain Scientific Rigor:** Ground all learning algorithms and cognitive assessments in established cognitive psychology research. Ensure that the spaced repetition implementation faithfully represents the SM-2 algorithm while adding enhancements based on contemporary research in educational technology.

2 Literature Review

The development of Memora is grounded in extensive research from cognitive psychology, educational technology, and computer science. This section examines the foundational theories and empirical studies that inform our approach.

2.1 Spaced Repetition Research

Spaced repetition has been extensively studied since the pioneering work of Hermann Ebbinghaus in the 1880s. Ebbinghaus's forgetting curve demonstrates that memory retention decreases exponentially over time, but this decline can be significantly mitigated through strategically spaced reviews.

Historical Foundation:

Hermann Ebbinghaus's seminal work "Memory: A Contribution to Experimental Psychology" (1885) established the mathematical relationship between time and forgetting. Through rigorous self-experimentation with nonsense syllables, Ebbinghaus demonstrated that:

- Memory retention follows a logarithmic decay function
- The rate of forgetting is steepest immediately after learning
- Strategic review can counteract the forgetting curve
- Spaced practice is superior to massed practice for long-term retention

This foundational work established the theoretical basis for all modern spaced repetition systems.

Key Findings from Literature:

1. **Cepeda et al. (2006):** Conducted a comprehensive meta-analysis of distributed practice effects, analyzing 317 experiments across 185 published studies. Their findings definitively showed that spaced practice consistently outperforms massed practice across various learning domains, retention intervals, and subject populations. The optimal spacing gap depends on the desired retention interval, following an approximate ratio of 10-20% of the retention period.
2. **SuperMemo Research:** The SM-2 algorithm, developed by Dr. Piotr Wozniak in 1988, represents a practical implementation of spaced repetition principles. Wozniak's research at the University of Technology in Poznan, Poland, led to the creation of SuperMemo software, which has been validated through decades of use by millions of learners. The algorithm's success lies in its mathematical approach to modeling forgetting curves and optimizing review intervals through an ease factor that adapts to individual item difficulty.
3. **Modern Adaptations:** Contemporary research has focused on personalizing spacing algorithms to account for individual differences. Studies by Lindsey et al. (2014) demonstrated that adaptive algorithms that consider individual learner characteristics outperform fixed-interval approaches. Research in machine learning applications to spaced repetition (Reddy et al., 2016) has shown promise in further optimizing review schedules through predictive modeling.

4. **Cognitive Load Theory:** Work by Sweller (1988, 2011) on cognitive load theory provides additional context for why spaced repetition works. By distributing cognitive load over time and reviewing material when it’s partially forgotten, spaced repetition systems maintain optimal cognitive load levels that promote deep learning without overwhelming working memory.

2.2 Cognitive Assessment Frameworks

Our MemScore evaluation system draws inspiration from established cognitive testing methodologies used in neuropsychology and educational assessment.

Memory Assessment:

Working memory assessment has a rich history in cognitive psychology, dating back to the work of Baddeley and Hitch (1974) on the multi-component model of working memory. Our card-matching paradigm is based on similar tasks used in the Wechsler Memory Scale and other standardized cognitive assessments. Research shows that working memory capacity is a strong predictor of learning ability and academic performance (Alloway & Alloway, 2010).

The emoji-based matching game in Memora adapts these principles for web-based delivery while maintaining psychological validity. The 10-second preview phase engages encoding processes, while the matching phase tests retrieval and recognition memory.

Spatial Memory:

The tile recall test is inspired by Corsi block-tapping tasks, first developed by Corsi (1972) for assessing visuospatial working memory. This type of assessment has been validated extensively in neuropsychological research and is known to engage different cognitive processes than verbal memory tasks.

Our progressive difficulty approach (3→5→7→9→11 tiles) allows for assessment across a wide range of spatial memory capacity, from typical adult performance (around 7 ± 2 items, consistent with Miller’s “magical number seven”) to exceptional performance levels.

Processing Speed:

Timed arithmetic operations reflect established measures of cognitive efficiency used in standardized tests such as the Wechsler Adult Intelligence Scale (WAIS) and the Woodcock-Johnson Tests of Cognitive Abilities. Processing speed has been shown to correlate with overall cognitive ability and learning efficiency (Kail & Salthouse, 1994).

The 30-second rapid-fire format creates time pressure that reveals true processing capacity rather than problem-solving ability, making it suitable for quick cognitive screening.

Psychometric Considerations:

While Memora’s MemScore is not intended as a clinical assessment tool, it follows established psychometric principles:

- Multiple cognitive domains measured independently
- Quantified scoring on a normalized scale (0-100)
- Reproducible procedures for test administration
- Clear scoring algorithms without subjective judgment

2.3 User Interface and Learning UX

Research in human-computer interaction for educational technology emphasizes several key principles implemented in Memora:

1. **Minimal Distraction:** Studies show that visual clutter increases cognitive load and reduces learning efficiency (Sweller, 2011). The principle of split-attention effect suggests that learners perform better when related information is integrated rather than separated. Memora’s cyber-grid design with dark theme minimizes visual noise while maintaining professional aesthetics. The decision to eliminate scrollbars and design for perfect viewport fit reduces unnecessary interface elements that could distract from learning content.
2. **Gamification Elements:** Research by Deterding et al. (2011) on gamification in education demonstrates that game-like elements can increase motivation and engagement without undermining learning effectiveness. Progress tracking, achievement streaks, and visual feedback have been shown to improve long-term usage and learning outcomes (Hamari et al., 2014). Memora implements evidence-based gamification through:
 - Study streak tracking for habit formation
 - Visual progress indicators for motivation
 - Performance metrics that provide meaningful feedback
 - Achievement systems that encourage consistent practice
3. **Personalization:** Adaptive interfaces that respond to user preferences and performance patterns significantly improve long-term usage and learning outcomes (Park & Choi, 2014). Research in adaptive educational systems shows that learners who receive personalized content and pacing achieve better outcomes than those using fixed approaches (Vandewaetere et al., 2011).
4. **Cognitive Affordances:** Norman’s (1988) concept of affordances in design is particularly relevant for educational interfaces. Interface elements should clearly communicate their function and relationship to learning goals. Memora’s topic cards, review buttons, and progress indicators are designed with clear visual hierarchies that guide user actions.
5. **Feedback Timing:** Research on feedback in learning (Hattie & Timperley, 2007) indicates that immediate feedback is generally superior to delayed feedback for procedural learning. Memora provides instant feedback after topic reviews and cognitive assessments to reinforce learning behaviors.

2.4 Technical Implementation Considerations

The technical architecture of Memora reflects current best practices in web application development:

MERN Stack: The combination of MongoDB, Express.js, React, and Node.js represents a modern, JavaScript-based full-stack approach that has gained widespread adoption for educational platforms. This stack provides:

- Unified language across frontend and backend (JavaScript)
- JSON-native data storage matching application data structures
- Component-based UI development for maintainability
- Scalable architecture suitable for growing user bases

Real-time Updates: Modern web technologies enable instant feedback and progressive enhancement of the learning experience. Research shows that system responsiveness significantly impacts user satisfaction and learning effectiveness (Nielsen, 1993).

Security and Privacy: Implementation of JWT authentication and secure data handling aligns with educational data privacy standards such as FERPA (Family Educational Rights and Privacy Act) in the United States and GDPR (General Data Protection Regulation) in Europe. Educational platforms must maintain high security standards due to the sensitive nature of student data.

Database Design: NoSQL databases like MongoDB offer flexibility for educational applications where data structures may evolve as features are added. The document-oriented model aligns well with object-oriented application design, reducing impedance mismatch between application code and data storage.

API Architecture: RESTful API design provides a clear, standardized interface between frontend and backend, facilitating future expansion to mobile applications or third-party integrations.

This literature review demonstrates that Memora’s approach is well-supported by empirical research and established best practices in educational technology, cognitive psychology, and software engineering. The platform synthesizes insights from multiple disciplines to create a comprehensive learning solution grounded in scientific evidence.

3 System Design

3.1 System Architecture

Memora follows a modern full-stack architecture with clear separation of concerns between frontend and backend components. The system is designed for scalability, maintainability, and optimal performance.

Architecture Overview:

The system implements a three-tier architecture pattern that separates presentation, business logic, and data management into distinct layers. This separation provides several advantages:

- **Independent Scaling:** Frontend and backend can be scaled independently based on demand
- **Technology Flexibility:** Each layer can use the most appropriate technologies without constraints from other layers
- **Maintainability:** Changes to one layer have minimal impact on others, reducing coupling
- **Development Efficiency:** Teams can work on different layers simultaneously

Frontend Layer:

React-based single-page application (SPA) built with Vite for optimal development experience and build performance. The frontend handles all user interface logic, state management, and client-side routing. Key characteristics include:

- Component-based architecture promoting reusability and maintainability
- Tailwind CSS utility-first styling for rapid UI development
- Framer Motion for smooth, performant animations
- React Context API for global state management (authentication, timer sessions)
- React Router for client-side navigation without page reloads
- Responsive design adapting seamlessly from mobile to desktop viewports

Backend Layer:

RESTful API server built with Express.js and Node.js, providing secure endpoints for data management, user authentication, and business logic implementation. The backend serves as the single source of truth for application data and enforces business rules. Architecture features:

- Express.js middleware chain for request processing (authentication, validation, error handling)
- Modular route organization by feature (auth, user, topics, doctags, journal)
- JWT-based stateless authentication enabling horizontal scaling
- Input validation using express-validator for security and data integrity
- Helmet.js security headers for protection against common web vulnerabilities
- CORS configuration for controlled cross-origin access

Database Layer:

MongoDB with Mongoose ODM provides flexible, schema-based data storage and efficient querying of user data, topics, and performance metrics. Key aspects:

- Document-oriented storage matching JavaScript object structures

- Mongoose schemas providing validation and type safety at the database level
- Indexed fields optimizing common query patterns (user lookups, review date sorting)
- Embedded documents for related data reducing join operations
- Atomic operations for transactional updates of spaced repetition state

Communication Layer:

JWT-based authentication with refresh token rotation ensures secure, stateless communication between frontend and backend:

- Access tokens (15-minute expiry) for API authorization
- Refresh tokens (7-day expiry) for obtaining new access tokens
- Token rotation on refresh for security
- Automatic token refresh in ApiService for seamless user experience
- User-specific localStorage cleanup on logout preventing data leakage

System Components

Frontend Module

- React 19.1.0
- Vite 7.0.4
- Tailwind CSS
- Framer Motion
- React Router

Backend Module

- Express.js 4.18.2
- MongoDB + Mongoose
- JWT Authentication
 - bcrypt Security
- Multer File Uploads

Core Features

- Spaced Repetition
- MemScore Assessment
- User Management
- Performance Analytics
- File Attachments

Security Layer

- JWT Token Rotation
- Password Hashing
- Input Validation
- CORS Configuration
- Helmet.js Headers

Data Management

- Topic CRUD Operations
- Review Scheduling
- Performance Tracking
 - User Preferences
 - Study Statistics

UI/UX System

- Cyber-Grid Theme
- Responsive Design
- Motion Animations
- Component Library
- Interactive Back-grounds

3.2 Database Schema Design

The database schema is designed to efficiently store user information, learning materials, and performance metrics while supporting complex queries for the spaced repetition algorithm.

User Collection:

The User collection stores authentication credentials, cognitive assessment results, preferences, and study statistics. Key fields include:

- **Authentication:** username, email, hashed password (bcrypt with 12 salt rounds)
- **Cognitive Profile:** memScore (0-100), hasCompletedEvaluation, evaluationResults (memoryGame, tileRecall, processingSpeed scores)
- **Preferences:** colorTheme, defaultDifficulty, retentionSpeed, memScoreRecalibrationFreq
- **Study Statistics:** currentStreak, longestStreak, totalStudyDays, lastStudyDate
- **Session Management:** refreshTokens array with token and expiration date
- **Timestamps:** createdAt, updatedAt (automatic via Mongoose)

Topic Collection:

The Topic collection is the core of the spaced repetition system, containing learning materials and scheduling metadata:

- **Content:** title (max 200 chars), content (max 10,000 chars), difficulty (1-5 scale)
- **Organization:** category, tags array for filtering and search
- **Spaced Repetition Fields:** easeFactor (default 2.5), interval (days), repetitions, nextReviewDate, isLearning flag
- **Performance Tracking:** reviewCount, averagePerformance (0-1 scale), lastReviewed timestamp
- **Attachments:** embedded documents with filename, mimetype, size, URL
- **External Links:** array of objects with title, URL, type, description
- **Ownership:** userId reference to User collection

Revision History Collection:

Tracks detailed information about each review session:

- **References:** userId and topicId for relationship tracking
- **Timing:** revisionDate, scheduledFor, actualRevision, wasOnTime flag
- **Performance:** performanceRating (1-5), timeSpent (minutes)
- **Notes:** optional user notes about the review session

Spaced Repetition Schedule Collection:

Maintains the current state of the SM-2 algorithm for each topic:

- **References:** userId and topicId
- **Algorithm State:** currentInterval, easeFactor, repetitions, nextReview
- **Status:** isActive flag for paused topics
- **Timestamps:** createdAt, updatedAt

Indexing Strategy:

To optimize query performance, strategic indexes are created on:

- User: `{ email: 1 }, { username: 1 }` for authentication
- Topic: `{ userId: 1, createdAt: -1 }` for user topic lists
- Topic: `{ userId: 1, nextReviewDate: 1 }` for due topics query
- Topic: `{ title: 'text', content: 'text' }` for full-text search
- RevisionHistory: `{ userId: 1, topicId: 1, revisionDate: -1 }` for history queries

3.3 API Endpoint Specifications

The RESTful API follows standard HTTP conventions with JSON request/response bodies.

Authentication Endpoints (`/api/auth/`):

- **POST `/register`:** Create new user account
 - Request: `{ username, email, password }`
 - Validation: username 3-30 chars alphanumeric, email format, password min 8 chars with mixed case and numbers
 - Response: `{ success, message, data: { userId, username, email } }`
- **POST `/login`:** Authenticate user
 - Request: `{ email, password }`

- Response: `{ success, data: { user, accessToken, refreshToken } }`
- **POST /logout**: Invalidate refresh token
 - Request: `{ refreshToken }`
 - Response: `{ success, message, data: null }`
- **POST /refresh**: Refresh access token
 - Request: `{ refreshToken }`
 - Response: `{ success, tokens: { accessToken, refreshToken } }`
- **GET /verify**: Verify current token and get user info
 - Headers: `Authorization: Bearer <accessToken>`
 - Response: `{ success, user: { id, username, email, memScore, ... } }`

User Endpoints (/api/user/):

- **GET /profile**: Get user profile
 - Response: `{ success, user: { ... } }`
- **PUT /profile**: Update user profile
 - Request: `{ preferences: { colorTheme, defaultDifficulty, retentionSpeed } }`
 - Response: `{ success, user: { ... } }`
- **POST /evaluation**: Save MemScore evaluation results
 - Request: `{ memoryGame, tileRecall, processingSpeed, overallScore }`
 - Response: `{ success, memScore: number }`
- **POST /study-session**: Record study session for streak tracking
 - Response: `{ success, currentStreak, longestStreak, totalStudyDays, isNewRecord }`

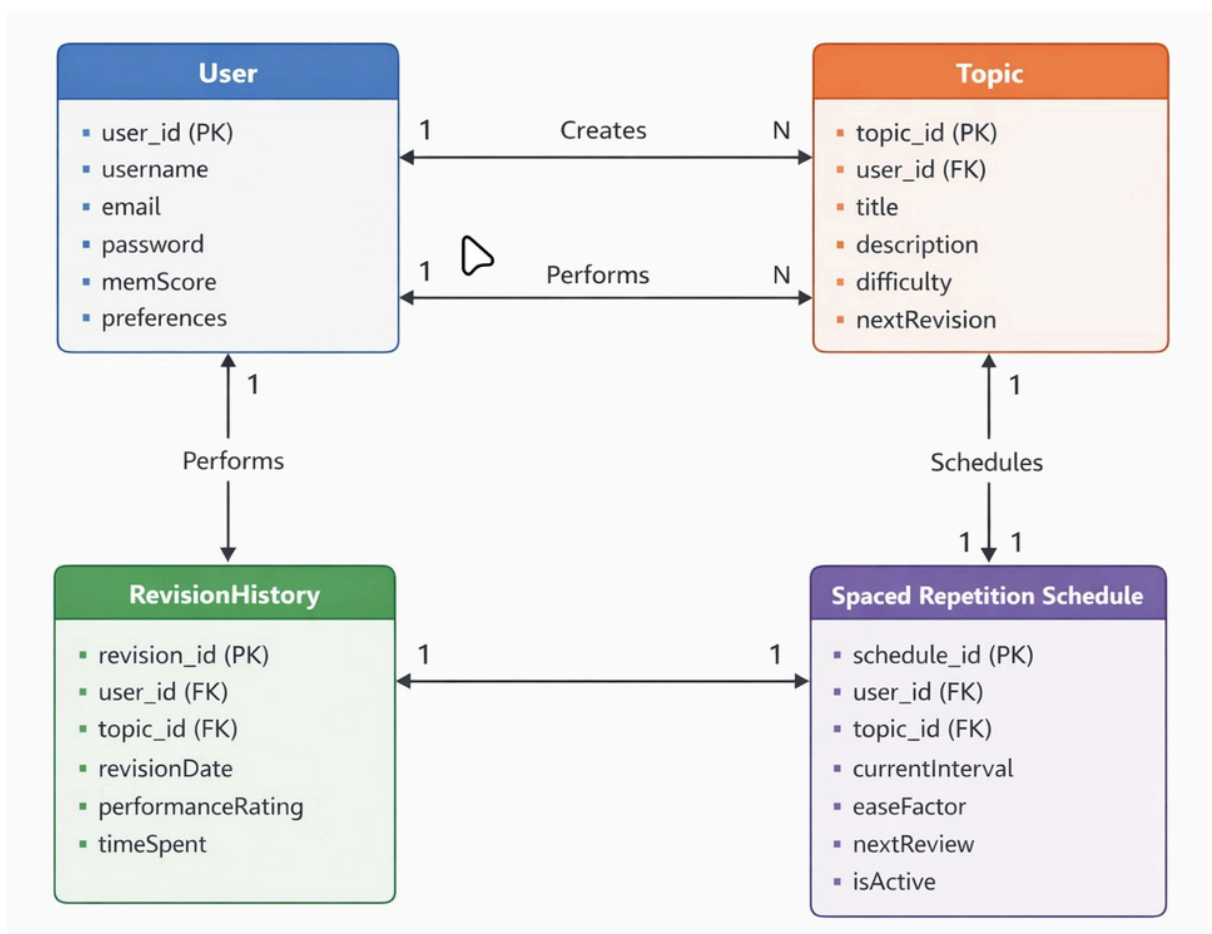
Topics Endpoints (/api/topics/):

- **GET /**: Get user's topics with filtering
 - Query: `?category=Science&difficulty=3&search=keyword&limit=50&page=1`
 - Response: `{ success, topics: [...], pagination: { page, limit, total, pages } }`
- **GET /due**: Get topics due for review today
 - Query: `?limit=10`
 - Response: `{ success, topics: [...], todaysCount, overdueCount, count }`
- **GET /upcoming**: Get upcoming topics for review
 - Query: `?days=7&limit=20`
 - Response: `{ success, topics: [...], count }`
- **POST /**: Create new topic
 - Request: `{ title, content, difficulty, category, tags, externalLinks, attachments }`
 - Response: `{ success, topic: {...}, crowdingPrevention: {...} }`
- **POST /:id/review**: Submit topic review with performance rating
 - Request: `{ quality: 1-5, responseTime?: number }`
 - Response: `{ success, topic: { nextReviewDate, interval, easeFactor, repetitions }, crowdingPrevention }`
- **POST /:id/skip**: Skip topic for today (postpone by 1 day with collision avoidance)

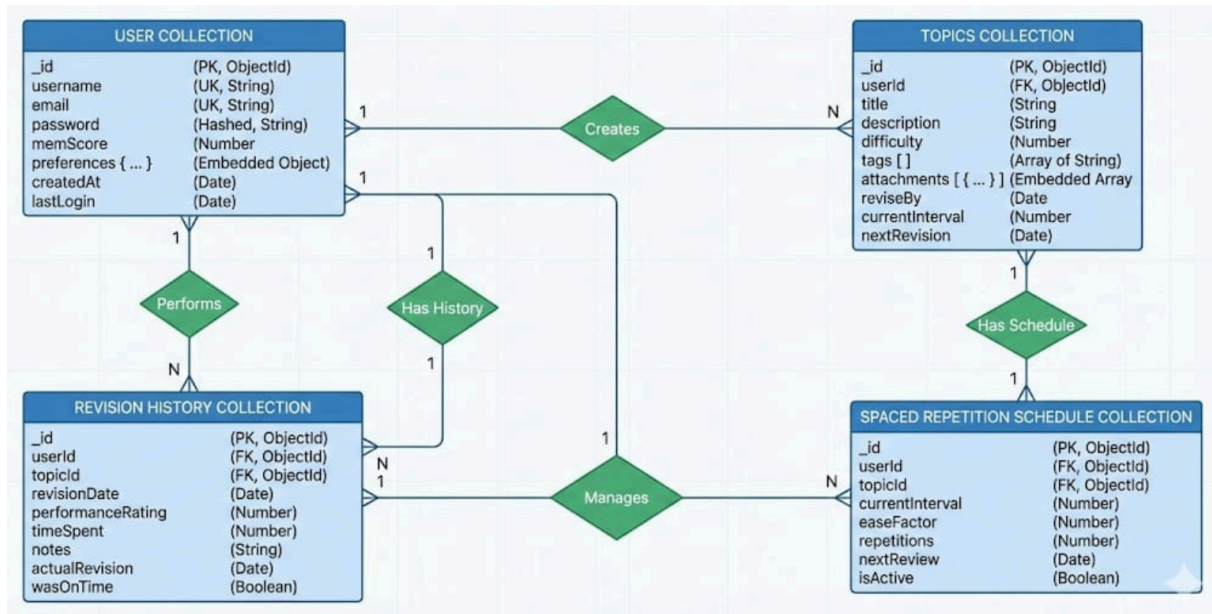
- Response: `{ success, message, topic: { nextReviewDate } }`
- **POST /prevent-crowding**: Redistribute topics to prevent crowding
 - Request: `{ targetDate: ISO date string }`
 - Response: `{ success, redistributed, count, details: [ ... ] }`
- **GET /workload**: Get daily topic workload for crowding analysis
 - Query: `?days=14`
 - Response: `{ success, workload: [{ date, count, averageDifficulty, crowdingLevel, isCrowded, thresholds }] }`

3.4 Entity-Relationship Diagrams

3.4.1 Simplified ER Diagram



3.4.2 Detailed ER Diagram



4 Implementation Details

4.1 SM-2 Spaced Repetition Algorithm

The core of Memora's learning system is an enhanced implementation of the SuperMemo SM-2 algorithm, adapted for web-based delivery with additional features for crowding prevention and difficulty-based load balancing.

Algorithm Foundation:

The SM-2 algorithm uses an ease factor to model how easily a user remembers each topic. After each review, the algorithm updates the ease factor based on the user's self-reported performance quality (0-5 scale), then calculates the optimal interval before the next review.

Mathematical Implementation:

```
// Quality ratings: 0-5 scale
// 0 = complete blackout, 5 = perfect response

function updateSpacedRepetition(quality) {
  this.reviewCount += 1;
  this.lastReviewed = new Date();

  if (quality < 3) {
    // Reset learning for poor performance
    this.repetitions = 0;
    this.interval = 1;
    this.isLearning = true;
  } else {
    // Update ease factor using SM-2 formula
    this.easeFactor = Math.max(
      1.3,
      this.easeFactor + (0.1 - (5 - quality) * (0.08 + (5 - quality) * 0.02))
    );

    // Calculate next interval
    if (this.repetitions === 0) {
      this.interval = 1; // First successful review: 1 day
    } else if (this.repetitions === 1) {
      this.interval = 6; // Second successful review: 6 days
    } else {
      this.interval = Math.round(this.interval * this.easeFactor);
    }

    this.repetitions += 1;

    // Mark as learned after successful repetitions
    if (this.repetitions ≥ 3 && quality ≥ 4) {
      this.isLearning = false;
    }
  }
}
```

```

// Set next review date
this.nextReviewDate = new Date(
  Date.now() + this.interval * 24 * 60 * 60 * 1000
);

// Update average performance (running average)
const totalPerformance = (this.averagePerformance * (this.reviewCount - 1))
  + (quality / 5);
this.averagePerformance = totalPerformance / this.reviewCount;
}

```

Key Parameters:

- **Ease Factor (EF):** Initial value 2.5, minimum 1.3. Higher EF means longer intervals.
- **Interval:** Days until next review. Increases multiplicatively with each successful review.
- **Repetitions:** Count of consecutive successful reviews (quality ≥ 3).
- **isLearning:** True until user demonstrates mastery (≥ 3 repetitions with quality ≥ 4).

Interval Progression Example:

For a topic with default ease factor (2.5) and consistent good performance (quality = 4):

- After 1st review: 1 day
- After 2nd review: 6 days
- After 3rd review: 15 days (6×2.5)
- After 4th review: 38 days (15×2.5)
- After 5th review: 95 days (38×2.5)

4.2 Crowding Prevention Algorithm

To prevent overwhelming study sessions, Memora implements an intelligent topic redistribution system that considers both topic count and difficulty levels.

Difficulty-Based Thresholds:

```

const baseThresholds = {
  light: 2,    // 1-2 topics
  medium: 3,   // 3 topics
  heavy: 4,    // 4 topics
  crowded: 5   // 5+ topics (requires redistribution)
};

const difficultyMultipliers = {
  1: 1.4,     // Easy: can handle 40% more
  2: 1.2,     // Below Medium: can handle 20% more
  3: 1.0,     // Medium: base thresholds
  4: 0.8,     // Hard: reduce by 20%
  5: 0.6      // Expert: reduce by 40%
};

```

Redistribution Logic:

1. Analyze daily topic counts with difficulty weighting for the next 14 days
2. Identify crowded days (exceeding difficulty-adjusted thresholds)
3. For each crowded day:
 - Calculate maximum allowed topics based on average difficulty

- Select excess topics, prioritizing higher difficulty ones
 - Find alternative dates within ± 3 to $+7$ day range
 - Prefer dates with lower existing load
4. Update nextReviewDate for redistributed topics
 5. Increment rescheduleCount for tracking

Alternative Date Selection:

```
// For each candidate date
const loadScore = currentCount + (currentAvgDifficulty / 5);

// Sort alternatives by:
// 1. Priority (closer dates first)
// 2. Load score (less loaded first)
alternatives.sort((a, b) => {
  if (a.priority !== b.priority) return a.priority - b.priority;
  return a.loadScore - b.loadScore;
});
```

4.3 MemScore Cognitive Assessment

The MemScore evaluation system comprises three distinct cognitive tests, each measuring different aspects of cognitive function.

Test 1: Memory Game

Card matching task measuring visual working memory and recognition:

- Grid size: 4×4 (16 cards, 8 pairs)
- Preview phase: 10 seconds showing all cards
- Scoring: `score = max(0, 10 - wrongAttempts)`
- Skills tested: Encoding, pattern recognition, spatial memory

Test 2: Tile Recall

Progressive spatial memory test measuring sequential memory capacity:

- Grid size: 5×5 (25 tiles)
- Rounds: 5 with increasing difficulty
- Sequence lengths: $3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$ tiles per round
- Show times: $3000\text{ms} \rightarrow 2700\text{ms} \rightarrow 2400\text{ms} \rightarrow 2100\text{ms} \rightarrow 2800\text{ms}$ (extra time for final round)
- Scoring: 2 points per correct round, -0.5 penalty per mistake
- Maximum score: 10 points

Test 3: Processing Speed

Rapid arithmetic measuring cognitive efficiency:

- Duration: 30 seconds
- Operations: Addition, subtraction, multiplication
- Number range: 1-50 for addition/subtraction, 1-12 for multiplication
- Scoring: `(correctAnswers / totalQuestions) × 10`

Overall MemScore Calculation:

```
overallScore = Math.round(
  (memoryGameScore + tileRecallScore + processingSpeedScore) / 3
```

```
);  
// Result: Integer from 0-10, displayed as percentage (0-100) in UI
```

4.4 Authentication System

Memora implements a secure, stateless authentication system using JSON Web Tokens (JWT) with refresh token rotation.

Security Features:

1. **Password Hashing:** bcrypt with 12 salt rounds
2. **Token Expiration:** Access tokens expire after 15 minutes, refresh tokens after 7 days
3. **Token Rotation:** New refresh token issued on each refresh request
4. **Token Limit:** Maximum 5 refresh tokens per user (oldest removed when exceeded)
5. **Automatic Cleanup:** Expired tokens removed from database

JWT Payload Structure:

```
// Access Token Payload  
{  
  id: userId,  
  username: username,  
  email: email,  
  iat: issuedAt,  
  exp: expiresIn15Minutes  
}  
  
// Refresh Token Payload  
{  
  id: userId,  
  iat: issuedAt,  
  exp: expiresIn7Days  
}
```

Token Flow:

1. User authenticates → Server generates access + refresh tokens
2. Client stores tokens in localStorage
3. API requests include access token in Authorization header
4. When access token expires (401 response), client uses refresh token to obtain new pair
5. On logout, refresh token is invalidated server-side

4.5 Frontend Component Architecture

The React frontend follows a component-based architecture with clear separation between pages, reusable components, and utility modules.

Core Components:

- **CyberGrid.jsx:** Interactive animated background with mouse-tracking gradients
- **AuthContext.jsx:** Global authentication state with useReducer pattern
- **TimerContext.jsx:** Focus session timer with global state
- **ApiService:** Singleton class handling all API communication with automatic token refresh

Page Components:

| Page | Purpose | Key Features | |——|———|———| | Dashboard | Main interface | Topic management, review queue, 7-day preview | | MemScoreEvaluation | Cognitive assessment | Three-phase testing with scoring | | Topics | Topic library | CRUD operations, filtering, search | | FocusMode | Concentrated study | Timer, distraction-free interface | | Journal | Activity log | Daily entries, performance tracking | | Chronicle | Calendar view | Review schedule visualization | | Profile | User settings | Preferences, streak display |

State Management Pattern:

```
// AuthContext reducer pattern
const AUTH_ACTIONS = {
  LOGIN_START: 'LOGIN_START',
  LOGIN_SUCCESS: 'LOGIN_SUCCESS',
  LOGIN_FAILURE: 'LOGIN_FAILURE',
  LOGOUT: 'LOGOUT',
  SET_USER: 'SET_USER',
  SET_LOADING: 'SET_LOADING',
  CLEAR_ERROR: 'CLEAR_ERROR'
};

const authReducer = (state, action) => {
  switch (action.type) {
    case AUTH_ACTIONS.LOGIN_SUCCESS:
      return { ...state, user: action.payload.user, isAuthenticated: true };
    // ... other cases
  }
};
```

5 Testing Considerations

5.1 Unit Testing Areas

Key areas requiring unit test coverage:

1. **Spaced Repetition Algorithm:**
 - Ease factor updates for all quality ratings (0-5)
 - Interval calculations at different repetition counts
 - Reset behavior for quality < 3
 - isLearning flag transitions
2. **MemScore Calculations:**
 - Memory game scoring with various wrong attempt counts
 - Tile recall scoring across all rounds
 - Processing speed percentage calculations
 - Overall score averaging
3. **Authentication:**
 - Password validation rules
 - Token generation and verification
 - Refresh token rotation
 - Token expiration handling
4. **Crowding Prevention:**
 - Threshold calculations with different difficulty averages
 - Topic selection prioritization
 - Alternative date finding
 - Redistribution limits

5.2 Integration Testing

End-to-end flows requiring integration tests:

1. **Complete Authentication Cycle:**
 - Registration → Login → Token refresh → Logout
 - Invalid credential handling
 - Session persistence across page reloads
2. **Topic Lifecycle:**
 - Create → Review → Schedule update → Delete
 - Filtering and pagination
 - Search functionality
3. **MemScore Flow:**
 - All three tests in sequence
 - Score persistence to database
 - Dashboard update after completion

4. **Crowding Prevention:**

- Detection of crowded days
- Redistribution execution
- UI notification updates

5.3 Performance Metrics

Key performance indicators to monitor:

- **API Response Times:** Target < 200ms for standard queries
- **Database Query Performance:** Indexed queries < 50ms
- **Frontend Render Times:** First Contentful Paint < 1.5s
- **Animation Frame Rate:** Maintain 60fps for Framer Motion animations

6 Deployment Considerations

6.1 Environment Configuration

Production Environment Variables:

```
# Server
PORT=3001
NODE_ENV=production

# Database
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/memora

# Security
JWT_SECRET=<secure-random-256-bit-key>
JWT_REFRESH_SECRET=<different-secure-random-key>
BCRYPT_SALT_ROUNDS=12

# CORS
FRONTEND_URL=https://memora.yourdomain.com
```

6.2 Security Hardening

Production security measures:

1. **Enable Rate Limiting:** Uncomment rate limiter in server.js (100 requests per 15 minutes)
2. **Strict CORS:** Remove localhost from allowed origins
3. **HTTPS Only:** Enforce TLS for all connections
4. **Secure Headers:** Helmet.js enabled by default
5. **Environment Secrets:** Never commit secrets to version control

6.3 Scalability Considerations

Architecture supports horizontal scaling:

- **Stateless Backend:** JWT authentication enables multiple server instances
- **Database Indexing:** Strategic indexes on high-query fields
- **API Separation:** Frontend and backend can be deployed independently
- **CDN Integration:** Static assets can be served from CDN

7 Future Enhancements

7.1 Planned Features

1. **ReviseBy:** Deadline management system for exam preparation
 - Set target dates for mastery
 - Automatic schedule optimization
 - Progress tracking toward deadlines
2. **Chronicle Enhancement:** Full calendar view for review scheduling
 - Drag-and-drop rescheduling
 - Weekly/monthly views
 - Integration with external calendars
3. **DocTags System:** Enhanced attachment capabilities
 - PDF document viewing
 - YouTube video embedding
 - Google Drive integration
 - Automatic content extraction
4. **Difficulty Matrix:** Smart topic load balancing
 - Visual difficulty distribution
 - Automatic rebalancing suggestions
 - Performance-based difficulty adjustment
5. **Performance Analytics:** Advanced progress tracking
 - Learning curve visualization
 - Retention rate graphs
 - Comparative performance metrics

7.2 Technical Improvements

1. **Testing Suite:** Comprehensive Jest + React Testing Library coverage
2. **Performance Optimization:** React.memo, useMemo, code splitting
3. **Offline Support:** Service worker for offline functionality
4. **Mobile Application:** React Native cross-platform app
5. **Real-time Sync:** WebSocket for multi-device synchronization

8 Conclusion

Memora represents a comprehensive solution to the challenges of effective learning and memory retention. By combining the scientifically-validated SM-2 spaced repetition algorithm with modern web technologies and innovative features like cognitive assessment and crowding prevention, the platform provides a personalized, intelligent learning experience.

Key Achievements:

1. **Scientific Foundation:** Implementation of the SM-2 algorithm with enhancements for real-world usage
2. **Cognitive Assessment:** Multi-faceted MemScore evaluation providing personalized learning profiles
3. **Intelligent Scheduling:** Crowding prevention with difficulty-weighted load balancing
4. **Modern Architecture:** Scalable MERN stack with secure authentication and responsive design
5. **User Experience:** Cyber-grid aesthetic with smooth animations and distraction-free learning

Technical Highlights:

- Full-stack JavaScript implementation with React 19 and Express.js
- MongoDB database with Mongoose ODM and strategic indexing
- JWT authentication with refresh token rotation
- Responsive design with Tailwind CSS and Framer Motion animations
- RESTful API with comprehensive input validation

The project demonstrates the practical application of cognitive science research to educational technology, providing a foundation for continued development and enhancement. Future iterations will expand functionality while maintaining the core focus on scientifically-backed learning optimization.

Impact Statement:

Memora has the potential to significantly improve learning outcomes for students and professionals by automating the complex task of review scheduling while providing personalized feedback and motivation. By making spaced repetition accessible and engaging, the platform addresses a critical gap between what cognitive science tells us works and what tools are available for everyday learners.

9 Gallery



Memora

Where memory science meets your daily study

Learn smarter, not harder with **memory-powered learning.**

Master any subject with scientifically-proven spaced repetition. Transform how you learn, remember, and retain knowledge with personalized study schedules that adapt to your progress.

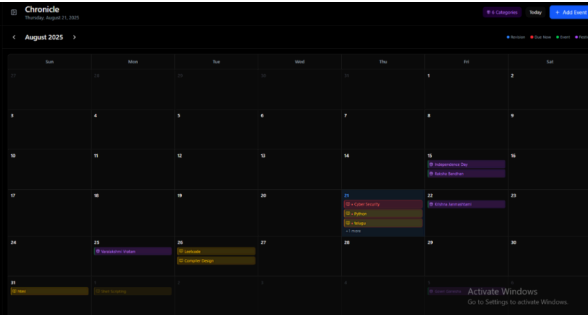
[Start Learning Free](#) [Get a Demo](#)

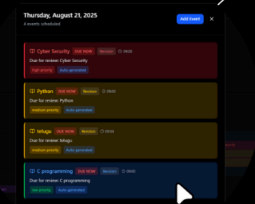
Activate Windows
Go to Settings to activate Windows.

Powered by Neuro Engine

CHRONICLE

Visualizes all upcoming revisions and deadlines in one place. Lets users view, manage, and adjust their revision schedule just like Google Calendar, but tailored for learning.





Functionality:

- Integrated View
- Adaptive Planning
- Future Insight

Today's Revision

4 due today

Cyber Security

🟡 Difficulty: 5/5 ⌚ Due: 21/08/2025

🟢 Done ⏸ Skip ✎ Edit 🗑 Delete

Python

🟡 Difficulty: 3/5 ⌚ Due: 21/08/2025

🟢 Done ⏸ Skip ✎ Edit 🗑 Delete

telugu

🟡 Difficulty: 3/5 ⌚ Due: 21/08/2025

🟢 Done ⏸ Skip ✎ Edit 🗑 Delete

C programming

🟡 Difficulty: 2/5 ⌚ Due: 21/08/2025

🟢 Done ⏸ Skip ✎ Edit 🗑 Delete

Upcoming Revision

5 total (showing 4)

Leetcode

🟡 Diff: 3/5 ⌚ Due: 26/08/2025

🔄 Review ⏸ Skip ✎ Edit 🗑 Delete

Compiler Design

🟡 Diff: 3/5 ⌚ Due: 26/08/2025

🔄 Review ⏸ Skip ✎ Edit 🗑 Delete

html

🟡 Diff: 3/5 ⌚ Due: 31/08/2025

🔄 Review ⏸ Skip ✎ Edit 🗑 Delete

Shell Scripting

🟡 Diff: 3/5 ⌚ Due: 01/09/2025

🔄 Review ⏸ Skip ✎ Edit 🗑 Delete

View All 5 Upcoming Topics

Memora

Dashboard

DocTags

Journal

Analytics

Chronicle

Settings

QUICK ACTIONS

Edit Entry

Weekly View

Monthly View

Journal

Thursday, August 21, 2025

Daily

Weekly

Monthly

Journal Entry

Edit

< Tuesday, August 19, 2025 >

Today 19-08-2025

Learning Journal - Tuesday, August 19, 2025

Study Summary

Topics Reviewed: 2

Focus Sessions: 0

Total Study Time: 0 minutes

Today's Activities

[07:22 PM] Started learning session

[07:22 PM] Reviewed "scripting" (just notes scripting) - Good

[07:22 PM] Reviewed "html" (basics) - Good

Reflections

What did I learn today?

Activate Windows
Go to Settings to activate Windows.

30

10 References

- Unbounded Human Learning: Optimal Scheduling for Spaced Repetition arXiv:1602.07032 [cs.AI]
- Amazon mechanical turk. <https://www.mturk.com>, 2005.
- Duolingo. <https://www.duolingo.com>, 2011.
- Spaced repetition. <http://www.gwern.net/Spaced%20repetition>, 2016.
- N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological bulletin*, 132(3):354, 2006.
- React. <https://reactjs.org>.
- Vite. <https://vitejs.dev>.
- Tailwind CSS. <https://tailwindcss.com>.
- Framer Motion. <https://www.framer.com/motion/>.

11 Appendix A: Technology Stack Summary

11.1 Frontend Technologies

Technology	Purpose	Version
React	UI Framework - Component-based architecture	19.1.0
Vite	Build Tool - Fast development server and bundling	7.0.4
Tailwind CSS	Styling - Utility-first CSS framework	4.1.11
Framer Motion	Animations - React animation library	12.23.6
React Router	Navigation - Client-side routing	7.7.0
Axios	HTTP Client - API communication	1.10.0
Lucide React	Icons - Consistent icon library	0.525.0
Marked	Markdown - Content rendering	16.1.2

11.2 Backend Technologies

Technology	Purpose	Version
Express.js	Web Framework - HTTP server and routing	4.18.2
MongoDB	Database - Document-oriented NoSQL storage	Latest
Mongoose	ODM - Schema-based MongoDB modeling	8.16.4
jsonwebtoken	Authentication - JWT token handling	9.0.2
bcryptjs	Security - Password hashing	3.0.2
Helmet	Security - HTTP headers protection	8.1.0
express-validator	Validation - Input sanitization	7.2.1
Multer	File Upload - Multipart form handling	1.4.5-lts.1
CORS	Security - Cross-origin resource sharing	2.8.5

11.3 Development Tools

Tool	Purpose
ESLint	Code Quality - Linting and style enforcement
Nodemon	Development - Auto-restart on file changes
PostCSS	CSS Processing - Tailwind integration
Autoprefixer	CSS - Browser compatibility prefixes
Git	Version Control - Source code management

12 Appendix B: Database Schema Reference

12.1 User Collection Schema

```
{
  username: String (unique, 3-30 chars, alphanumeric + underscore),
  email: String (unique, lowercase, validated format),
  password: String (bcrypt hashed, excluded from queries),
  memScore: Number (0-10),
  preferences: {
    colorTheme: String ("monochrome" | "neon-blue" | "neon-green"),
    defaultDifficulty: Number (1-5),
    retentionSpeed: String ("fast" | "medium" | "slow"),
    memScoreRecalibrationFreq: Number (1-365 days)
  },
  hasCompletedEvaluation: Boolean,
  evaluationResults: {
    memoryGame: Number,
    tileRecall: Number,
    processingSpeed: Number,
    overallScore: Number,
    completedAt: Date
  },
  refreshTokens: [{ token: String, createdAt: Date, expiresAt: Date }],
  currentStreak: Number,
  longestStreak: Number,
  lastStudyDate: Date,
  totalStudyDays: Number,
  isActive: Boolean,
  createdAt: Date,
  updatedAt: Date
}
```

12.2 Topic Collection Schema

```
{
  title: String (max 200 chars),
  content: String (max 10000 chars),
  userId: ObjectId (ref: User),
  tags: [String],
  difficulty: Number (1-5),
  learnedDate: Date,
  category: String (Science|Mathematics|History|Language|Technology|Arts|
Business|Other),
  attachments: [{
    filename: String,
    originalName: String,
    mimetype: String,
    size: Number,
  }]
```

```

    url: String,
    uploadedAt: Date
  }],
  externalLinks: [{
    title: String,
    url: String,
    type: String (youtube|google_drive|notion|github|website|file|other),
    description: String,
    addedAt: Date
  }],
  isActive: Boolean,
  lastReviewed: Date,
  reviewCount: Number,
  averagePerformance: Number (0-1),
  easeFactor: Number (min 1.3, default 2.5),
  interval: Number (min 1 day),
  repetitions: Number,
  nextReviewDate: Date,
  isLearning: Boolean,
  rescheduleCount: Number,
  createdAt: Date,
  updatedAt: Date
}

```

12.3 Database Indexes

```

// User Collection Indexes
{ email: 1 }           // Login queries
{ username: 1 }        // Username lookups
{ createdAt: -1 }      // Chronological sorting

// Topic Collection Indexes
{ userId: 1, createdAt: -1 } // User topic lists
{ userId: 1, nextReviewDate: 1 } // Due topics queries
{ userId: 1, isActive: 1 }    // Active topic filtering
{ userId: 1, category: 1 }    // Category filtering
{ tags: 1 }                   // Tag-based queries
{ title: 'text', content: 'text' } // Full-text search

```

13 Appendix C: Project Statistics

13.1 Codebase Metrics

Metric	Value
Total Files	100 files
Frontend Components	20+ React components
Frontend Pages	13 page components
Backend Routes	5 route modules
Database Models	5 Mongoose schemas
API Endpoints	25+ endpoints
Lines of Code (Frontend)	8,000 lines
Lines of Code (Backend)	2,500 lines

13.2 Feature Coverage

Feature	Status	Description
User Authentication	Complete	JWT with refresh token rotation
Topic Management	Complete	Full CRUD with attachments
Spaced Repetition	Complete	SM-2 algorithm implementation
Cognitive Assessment	Complete	3-part MemScore evaluation
Crowding Prevention	Complete	Difficulty-based redistribution
Study Streaks	Complete	Daily tracking with records
Focus Mode	Complete	Timer and distraction-free UI
Journal	Complete	Activity logging and daily entries
Chronicle	Partial	Calendar view (basic)
Analytics	Partial	Basic performance metrics
DocTags	Partial	File attachment system
ReviseBy	Planned	Deadline management
Mobile App	Planned	React Native implementation
Offline Support	Planned	Service worker integration